

If I were mentoring a fresh FPGA graduate, my goal would **not** be to teach ARINC 818-2 directly first.

My goal would be:

Teach them a repeatable protocol-learning framework that works for ARINC 818-2, PCIe, Ethernet, SpaceWire, Aurora, JESD204, ARINC 429, MIL-STD-1553, CAN, UART, SPI, etc.

Most juniors fail because they start reading protocol documents from page 1 and drown in details. Professional FPGA engineers work differently.

The FPGA Protocol Mastery Framework

Every protocol should be studied in 8 phases.

Phase 0 : Build Foundations

Phase 1 : System Understanding

Phase 2 : Protocol Architecture

Phase 3 : Packet/Data Structure

Phase 4 : Timing and Link Layer

Phase 5 : Hardware Implementation

Phase 6 : Verification & Debug

Phase 7 : Product Integration

If you master this process once, you can learn any protocol later.

Phase 0 — Foundation Layer

Before ARINC 818, the junior must understand:

Digital Design

Must know:

- Flip flops
- FSM
- Counters
- Shift registers
- FIFOs
- CDC
- Reset synchronization

Without this:

Protocols will look like magic.

FPGA Architecture

Must know:

LUT

What it does

FF

Storage element

BRAM

Packet buffering

DSP

Processing

Clocking resources

MMCM

PLL

Data Communication Basics

Must understand:

Serial vs Parallel

Why modern protocols use serial links

Simplex

One direction

Half Duplex

Both directions but not simultaneously

Full Duplex

Both simultaneously

Latency

Throughput

Bandwidth

Determinism

Critical in avionics.

Phase 1 — Understand The System

Most people skip this.

Big mistake.

Before opening ARINC 818 document:

Ask:

Why does this protocol exist?

ARINC 818 exists because:

Aircraft displays require

- Huge video data
 - Low latency
 - High reliability
-

Ask:

What problem does it solve?

Without this:

Specification becomes meaningless.

For any protocol answer:

Who are the users?

What data travels?

Why is it needed?

What are constraints?

What fails if protocol fails?

Example:

ARINC 818

Users:

- Cockpit displays
- Mission computers
- Sensors

Data:

- Video

Constraint:

- Real time
-

At the end of Phase 1:

Junior should explain protocol in 2 minutes without looking at notes.

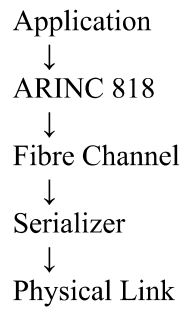
Phase 2 — Protocol Architecture

Now open specification.

Ignore details initially.

Draw architecture.

For ARINC 818:



Ask:

Which layers belong to protocol?

Which layers are inherited?

Which layers are physical?

Which layers are FPGA responsibility?

This is critical.

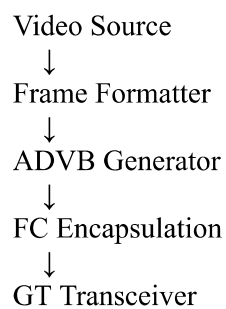
Many engineers waste months implementing things already provided by Xilinx IP.

For every protocol create:

Block Diagram

Mandatory.

Example:



Phase 3 — Packet Analysis

This is where real protocol learning begins.

Every protocol eventually moves data in a structure.

Could be:

- Frame
- Packet
- Word
- Symbol

Take one packet.

Dissect every bit.

Create table.

Example:

Field
Size
Purpose

Ask:

Who generates it?

Who consumes it?

Is it mandatory?

Is it configurable?

Does FPGA need to modify it?

For ARINC 818:

Learn:

FC Frame

SOF

EOF

Payload

CRC

Ordered Sets

Never memorize.

Understand purpose.

Exercise:

Take one packet.

Decode manually.

Bit by bit.

Phase 4 — Timing Layer

Most protocol failures happen here.

Not packet layer.

Timing layer.

Study:

Clock source

Link speed

Synchronization

Alignment

Timing constraints

For ARINC 818:

Need understanding of:

Fibre Channel line rates

8b/10b

or

64b/66b

depending on implementation

Must know:

Why encoding exists.

Not just what it is.

Study:

Running disparity

Clock recovery

Comma detection

Lane alignment

FPGA engineers spend most debugging time here.

Phase 5 — Hardware Implementation

Now think like FPGA engineer.

Not protocol engineer.

Break protocol into modules.

Example:

Frame Generator

Header Generator

Payload Generator

CRC Generator

FIFO

Flow Control

Transceiver Interface

Ask for each module:

Inputs?

Outputs?

Clock?

Reset?

State machine?

Buffer size?

Latency?

Now create:

RTL Architecture Document

Before writing RTL.

Professional rule:

1 hour architecture

10 minutes coding

Phase 6 — Verification

Fresh graduates underestimate verification.

Industry doesn't.

For each protocol:

Build:

Protocol Checker

Verifies:

- Header
 - Payload
 - CRC
-

Packet Generator

Generates legal packets.

Error Injector

Inject:

- Bad CRC
 - Missing SOF
 - Missing EOF
-

Scoreboard

Compare TX and RX.

Now protocol becomes understandable.

Phase 7 — Debugging

Most learning happens here.

Learn tools:

Vivado ILA

VIO

GT Debug

Protocol Analyzer

For every issue ask:

Is it

RTL

or

Timing

or

Link

or

Protocol

Develop debugging tree.

Phase 8 — Product Integration

This separates engineers from students.

Understand:

System interfaces

ARINC 818 may connect to:

- Video pipeline
 - DDR
 - DMA
 - Processor
 - Sensors
-

Questions:

Frame rate?

Resolution?

Link speed?

Buffer size?

Redundancy?

Fault handling?

Now protocol becomes product.

ARINC 818-2 Learning Roadmap Specifically

Once the framework is understood:

Stage 1

Learn:

- Aircraft display systems
 - Video concepts
 - Resolution
 - Frame rate
 - Pixel formats
-

Stage 2

Learn Fibre Channel basics

Because ARINC 818 is built on Fibre Channel.

Study:

- FC frame
 - SOF
 - EOF
 - CRC
 - Ordered sets
-

Stage 3

Learn ARINC 818 terminology

- ADVB
 - Container
 - Object 0
 - Object 1
 - ICD
-

Stage 4

Study ARINC 818 packet format

Bit level.

Stage 5

Study link establishment

Stage 6

Study redundancy concepts

Stage 7

Study complete ICD examples

This is what companies actually use.

Stage 8

Implement

1. ARINC 818 TX
2. ARINC 818 RX
3. Loopback
4. Video Pattern Generator
5. CRC Verification
6. Error Injection

Final Objective

By the end, the junior should be able to answer these questions for **any protocol**:

1. Why does the protocol exist?
2. What problem does it solve?
3. What is the architecture?
4. What is the data unit?
5. How is data formatted?
6. How is timing maintained?
7. How is it implemented in FPGA?
8. How is it verified?
9. How is it debugged?
10. How is it integrated into a product?

If they can answer those ten questions without opening the specification, they have moved from "reading a protocol" to "engineering with a protocol." That is the level expected of an FPGA engineer who can independently develop protocol-based projects.
